Evolutionary Algorithms: Final report

Tristan Toye (r0849537)

December 31, 2024

$if\ \_\_name\_\_ == \ 'main'$: must be added to the file from which the test is called. If not, a runtime error will occur as described in the multiprocessing documentation on **safe importing of the main module**. [*multiprocessing* n.d.]

1 Metadata

- **Group members during group phase:**Dag Malstaf and Noa Pauwels
- **Time spent on group phase:** 15 hours
- **Time spent on final code:** 75 hours
- **Time spent on final report:** 12 hours

2 Changes since the group phase

1. Added support in initialization for a **greedy initialisation** and **nearest-neighbour initialisation**. In addition, checks were added to ensure the quality of the initial population in terms of **diversity** and the **absence of invalid connections**.
2. The mutation was reworked to include Displacement mutation (**DM**), Simple inversion mutation (**SIM**) and Inversion mutation (**IVM**).
3. Changed the variation to include the order crossover (**OX1**), position-based crossover (**POS**), and Position based crossover [Larranaga et al. 1999]. In addition, a mechanism for the generation of offspring of the **pheromone matrix**[Marco Dorigo 1997 & Dorigo and Gambardella 1997] was implemented. Finally, a random **inversion mutation on converged candidates** was added to complete the offspring group.
4. In addition to the k-tournament elimination, controls were added to ensure enough diversity in the remaining population. An initial elimination of equals was implemented, after which a **penalty** was given to solutions that were **too similar** to each other.
5. Local search **3-opt** and **double bridge 4-opt** was added ass well as a **lineair local search operator** for the OX1, POS and Heuristic offspring. Iterated Lin-Kernighan was attempted, but due to the non-symmetric nature of the problems was voided. [David S. Johnson 1997]
6. A second **concurrent process** was added that uses local search operator 3-opt, inversion mutation and the pheromone matrix to produce quality results.
7. An extensive performance review was made, including **Numba** functions [*Numba documentation* n.d.]. Additionally, threading was considered for the recombination operators. However, the sequential implementation proved more performant due to **overhead and cache starvation**.

3 Final design of the evolutionary algorithm

3.1 The three main features

1. Firstly, the use of the **pheromone matrix** to create offspring has shown a significant impact on the convergence, both guiding towards local optima and also enabling the escape from one. As this matrix contains information from the entire generation and all previous generations, the hereditary information of offspring has shown to be significantly higher than those of the other recombination operators OX1, POS, and heuristic recombination. More details are in section 3.8.
2. Secondly, the **3-opt** local search operator is vital in guiding the algorithm towards relevant candidate solutions. The solutions provided by the local search operator empower other aspects, like mutation and variation, to contribute significantly more to finding the best approximated solution. Details in section 3.10.
3. Thirdly, the **random inversion mutation** is the key to escaping local minima. The current implementation of the 3-opt algorithm is unable to reverse the order of elements due to the non-symmetric nature of the problem. This random inversion enables the efficient escape from local minima. Details in section 3.7.

3.2 The main loop

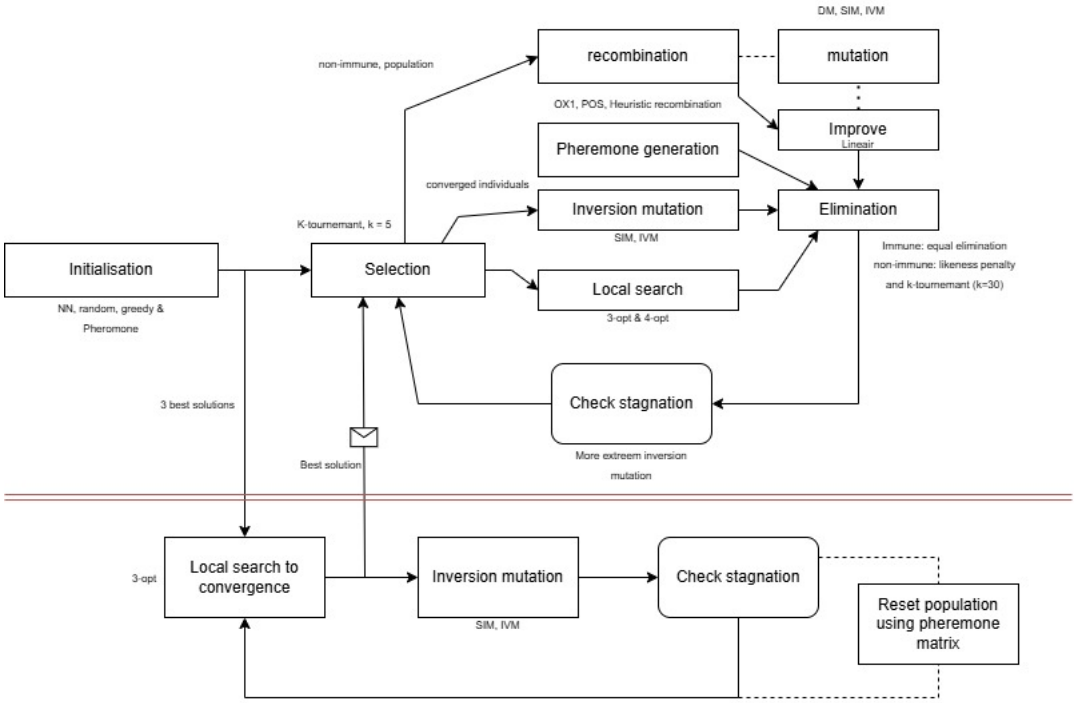

Figure 1: Diagram overview of the architecture of the genetic algorithm **genetic algo**

3.3 Representation

3.3.1 Adjacency representation or one-line representation

The adjacency representation comes in two variants: the one-line and two-line respectively:

$$\sigma_1 = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 1 & 4 & 2 \end{pmatrix} \quad \sigma_2 = \begin{pmatrix} 4 & 3 & 2 & 1 \\ 2 & 4 & 1 & 3 \end{pmatrix} \tag{1}$$

In one-line notation, the "natural" order $(1, 2, 3, ...)$ is assumed for the elements in the first row, which allows us to omit this row entirely. Given this assumption, σ_1 can be written simply as $\sigma_1 = \mathbf{3142}$.

3.4 Comparison with other representations

The adjacency representation ensures that distinct representations represent distinct Hamiltonian cycles. Each Hamiltonian cycle has multiple path notations, but **only one adjacency notation**. Moreover, it represents the genetic information most accurately. In the connection of cities, there is the most information to be found as these edges directly represent the cost of the path. Moreover, this **common sub-tours are easy to find** using the adjacency representation even when they don't occur in the same place in the cycle. Lastly, with the proper abstraction, the adjacency representation can be used in much the same way as the path representation **without any performance penalties**. The reverse transition, from the path to adjacency notation, does not come without a performance penalty, however. The **biggest downside** of the representation is the inability to traverse the cycle in the other direction easily. And, similarly, the **unability to easily reverse** a (sub)tour. To implement these functionalities, a transition to the path notation was made. Finally, checking the **cyclic requirement** of the tour is difficult at first glance; however, by abusing the abstraction of the path notation, we can use the easiness of the path notation cyclic check without any performance penalties.

An alternative representation is the path notation. The main advantage of this representation is that the permutation of the cities is guaranteed cyclic. This fact will be used in the **random initialisation**. Another representation is the **Ordinal representation**. The biggest benefit of the ordinal representation is the compatibility with traditional crossover operations. However, experiments have shown that the resulting offspring is of bad quality.[**(representation)**]

Other representations exist based upon **matrixes** were not explored.

3.5 Initialization

There are **four** major heuristics for the TSP problem [David S. Johnson 1997]:

1. Nearest Neighbours (NN)
2. Greedy
3. Clarke-Wright
4. Christofides

The paper mentioned that they proceeded to use NN to test the performance of the local search operators as it gave the best results. In accordance, we followed that recommendation. However, the NN heuristic has a significant downside of severely reducing the diversity of the initial population. To combat this, the greedy heuristic was also used to build part of the initial population. The rest of the population size was filled with random cycles. The initial population is built as follows to enhance quality and diversity:

1. 30% Nearest Neighbours (NN)
2. 40% Greedy
3. 30 % random

However, some restrictions were put in place to enhance the quality of the initial population. All candidates with a **likeness of over 95% are deleted** (one of them). This quality is measured in equal edges, which calculation is linear in time for the adjacency representation. Moreover, one can stop short once more than 5% of edges are different, significantly speeding up the process. There are bound to be some removals, so we use the **pheromone matrix to generate new individuals** (see section 3.15). This new population is again tested against the restrictions. This process is iterated until there are no removals due to the limits imposed.

Furthermore, another restriction is imposed: **there are no invalid edges in the initial population**. This restriction is imposed easily in the NN and greedy heuristic initialisation. For the random initialisation, we use the linear local search operator (more details in section 3.10) to remove the invalid edges; in a small percentage of cases, this does not suffice and we generate the random cycle again.

The **population size** was chosen to allow sufficient diversity within the population while maintaining performance. The population size of **80** was empirically chosen as well were the fractions of the different heuristics to ensure good quality of initial population.

3.6 Selection operators

A simple **K-tournament selection** scheme was used. As discussed in the lectures, it is performed in computational cost and offers the flexibility of adjusting the selection pressure. Some basic experiments were run with different values of k. Ultimately we settled on $k = 5$ as it showed a performant selection pressure. A more complex selection scheme with adaptivity through the process could still be explored. These experiments were not implemented in this project. No other selection schemes were implemented as the k tournament proved sufficient and solid.

3.7 Mutation operators

Multiple mutation operators were implemented based on the performance reviews outlined in [Larranaga et al. 1999]:

1. Displacement Mutation (DM) takes a subpath and inserts it elsewhere
2. Simple Inversion Mutation (SIM) inverses a subpath in place
3. Inversion Mutation (IVM) inverses a subpath and inserts it elsewhere

As discussed in section 3.10, the 3-opt problem cannot handle inversions due to the unsymmetric nature of the problem. This makes the SIM and IVM mutation paramount to introduce randomness and allow access to otherwise unreachable local optima. All of these mutation operators are defined by two indexes: the indexes that build the subpath of the operation. However, we reasoned that mutation of a longer sub-path length makes little sense as they can be accomplished as one of the many options of **iterated shorter inversions**. This iterated approach has the possibility to introduce even more randomness than one bigger inversion. Based on this notion, we restricted the freedom of the mutation by choosing only one index with a fixed length of the subpath.

3.8 Recombination operators

Multiple recombination operators were discussed in [Larranaga et al. 1999]. Following the performance analysis of the referenced paper, the decision was made to implement **OX1**, **POS** and **heuristic recombination**. Moreover, based upon further observations in experiments, **pheromone** and **inversion-based offspring generation** were

added to ensure diversity and allow the escape of local minima. The amount of offspring generated was empirically chosen based on the trade of between computational cost and contribution to diversity in the population:

1. 2: Order crossover (OX1)
2. 2: Position-based crossover (POS
3. 2: Heuristic recombination
4. 2*converged population: inversion mechanism
5. 10: pheromone mechanism

Order crossover (OX1) is based upon the heuristic that the absolute position of the cities does not contain any information, but their relative position does. OX1 aims to produce offspring while maintaining the relevant order of the cities. It takes a subpath from parent1, copies it to the child, and fills the remaining cities with the same order as the cities from parent2, starting from the index of the end of the subpath in parent1. This operator uses a parameter to control the length of the subcycle, which, after experiments, was set to 15% of the total number of cities. When the subpath between the indexes in both parents are equal, we recreate both parents. Otherwise, the operator produces new cycles. With less overlap more cities are getting a new neighbour as more cities become unavailable for allocation. [Larranaga et al. 1999]

Position-based crossover (POS) uses the same heuristic as OX1. However, instead of choosing a sequential subpath it takes independent indexes as fixed positions. The fixed positions are copied from parent1, and the other cities are inserted in the sequential order of parent2, starting from city 0. Likewise, the POS operator uses a parameter to control the number of fixed indexes; empirically, this was set to 15% of the total number of cities. POS has the same behaviour as OX1 when the overlap on indexes varies. [Larranaga et al. 1999]

Heuristic recombination uses the greedy heuristic to combine parents. Unlike OX1 and POS, it is not a position but edge-based, leaning perfectly towards the adjacency representation. It takes the edges of both parents leaving the last city of the sequence of the child. From the parents, it chooses the shortest edge that is still available to allocate. If both parent edges are not available for allocation, it takes the nearest available neighbour. So, unlike OX1 and POS, the heuristic recombination is aware of the problem. Moreover, it does not rely on any parameter except for the starting city, which is chosen randomly. How the cities are connected is highly dependent on the starting city; however, when the overlap decreases, more allocations come from the closest nearest neighbours.

The **inversion mechanism** (arity = 1) uses a candidate that has converged using the local search operators (see section 3.10), combined with the inversion mutations SIM and IVM operators(see section 3.7) to produce offspring. These inversion mutation were chosen since the local optimiser is unable to reverse city ordering (see section 3.10. Controlled by a parameter that increases when stagnation is detected, this operator mutates a copy of the converged solution multiple times. This offspring has shown great success in escaping local optima.

More information on the **pheromone mechanism** in section 3.15. These offspring have been shown to effectively produce guided diversity in the population that allows the finding of new local optima. As the pheromone matrix contains information from every individual of every generation of offspring, the arity of this mechanism is well above 2. We believe it is due to this broader scope of knowledge that this operator produces successful offspring.

We experimented with a **Common subcycle** heuristic: when two parents are decent candidates and share a common subtour, that subtour must be of good quality. However, this showed discouraging results and was not kept in the final project. Moreover, we believe the usage of multiple recombination operators allowed sufficient diversity in the offspring that new operators were not necessary.

3.9 Elimination operators

Elimination was done using **k-tournament** on the fitness values (section 3.11) of the relevant population (section 3.14) with $k = 30$, chosen emperically. This value of k is constant throughout the algorithm. We refer to future work to determine the impact of using a dynamic value for k.

No other elimination techniques were implemented. However, we did experiment with $k = population\_size$ which is equivalent to top lambda elimination. These experiments showed the K-tournament to be more performant overall.

3.10 Local search operators

The local search operator was based upon the comprehensive overview in [David S. Johnson 1997]. A case study was made based upon the findings in the referenced paper. We determined that the 3-opt, double bride 4-opt and lin-Kernighan had the most potential to be integrated within the genetic algorithm. However, all of the above mentioned techniques are for symmetrical problems. An attempt was made to make an adapted version of the Lin-Kernighan algorithm. However, the strength of the Lin-Kernighan algorithm lies in it performance to cost trade-off. Numerous assumptions were made based on the symmetrical nature of the problem to optimise this trade-off. After reworking the algorithm so as not to rely on these assumptions, the strengths of the Lin-Kernighan algorithm were eroded. With this observation, we implemented an adapted version of **3-opt** and **double bridge 4-opt**.

One could look at 2-opt, 3-opt and their variants as a neighbourhood exchange process. 2-opt breaks two edges and recombines them in all the possible ways, choosing the one with the most improvement if any. This improvement is calculated as

$$improvement = cost\_of\_new\_edges - cost\_of\_current\_edges$$

This is a deterministic and exhaustive process with severe computational cost. However, due to the non-symmetric problem, another problem arises: the cost of an inversed section is not the same as the original section. This means that the initial cost equation no longer holds and needs to include the cost of the inverted sub-tour:

$$improvement = (cost\_of\_current\_edges + \sum non\_inverted\_tour) - (cost\_of\_new\_edges \ + \sum inverted\_tour)$$

This makes the computational cost even higher. To add insult to injury, inversions are not possible using the adjacency representation. To alleviate this issue, we decided to focus only on swaps that do not need to inverse a section of the tour. This means that 2-opt is no longer possible, their is only one legal 3-opt swap and four 4-opt swaps. However, the literature was clear that 4-opt most often is not worth the computational cost, except for the double bride variant where two illegal 2-opt moves are made to build a legal cycle. Double bride 4-opt does not require inversion of the subtours. So there are two remaining legal local optimiser swaps:

1. 3-opt without inversion
2. Double bridge 4-opt

Moreover, a **linear local optimiser** was provided to improve the created offspring of the OX1, POS and heuristic recombination (see section 3.8). The primary objective of this optimiser is twofold: remove invalid edges at a low computational cost. This is a sequential variant of 3-opt, omitting the need to search for other edges while allowing all the 3-opt possibilities.

3.10.1 Computational cost

As mentioned, the computation cost is severe when running a 3-opt or double bridge 4-opt to convergence. However, as mentioned in the paper [David S. Johnson 1997] there are several ways to increase the performance of the local search operator. The parallel executions were not implemented as they relied on neighbourhood segmentation and experimental observations showed that the overhead of creating multiple threads would slow down the overall computation time. Instead, we focused on

1. NN-search for candidates
2. Don't look bits

Firstly, the **NN-search for candidates** is based on the heuristic that the best edge must be a close neighbour of the city. Instead of searching for all possibilities of edges to find a second edge, we focus on the closes k neighbours to find an edge. The third edge is found in a similar fashion. It was outlined in the paper that the penalty on the solution found was on average between $0.1\% - 0.2\%$ with the benefit of greatly reducing computational cost from $\mathcal{O}(n^3)$ to $\mathcal{O}(k^2 n)$ with k the maximum nearest neighbour search [David S. Johnson 1997]. In our implementation, based upon the recommendation of the paper, $k = 20$. No experiments were run to vary this parameter nor to make it dynamic through iterations. We refer to future work to determine the impact of such changes. A similar optimisation was made for the double bride 4-opt, reducing the run time from $\mathcal{O}(n^4)$ to $\mathcal{O}(k^2 n^2)$ with k the maximum nearest neighbour search.

Secondly, an observation was made by Bently as outlined in the paper [David S. Johnson 1997]. There is a significant chance that when the neighbours of a city did not change, there will be no improvement found when running the local search optimiser again. This is a heuristic, not a proven fact. However, empirical results have shown this heuristic to proven correct in almost all runs of the local search algorithm. Based upon this heuristic, we introduced a **don't loop bits** array. We set the bit corresponding with the city to $False$ when there is no optimisation found using the local search optimiser using the city as $t1$. We reset the don't look bits of all the cities that have a neighbour change when applying a local search swap. When running again, we skip all the cities with their don't look bits set to $False$ as options for city $t1$. These don't look bits were also implemented for double bridge 4-opt.

Using this implementation of 3-opt and double bridge 4-opt, we noticed a significant increase in the performance of the genetic algorithm. Without the local search optimiser the genetic algorithm was never able to come close to the local optima found using the local search operator. However, we noticed that the genetic component was able to improve the outcome of the local search even further. Furthermore, the reduction in possibilities of 2-opt, 3-opt and 4-opt results, by design, in the inability of the local search algorithm to inverse sequences within the tour. To overcome this shortcoming we rely on the genetic component of the algorithm to generate new individuals with different ordering of cities.

3.11 Diversity promotion mechanisms

We implemented a likeness penalty to ensure diversity as follows:

$$scaling* = \begin{cases} 1, & \text{if } equal\_edges \leq x\%\ total\_number\_cities \\ 2 - (2 * (1 - \frac{equal\_edges}{total\_number\_cities}))^{penalty}, & \text{otherwise} \end{cases}$$

with

$$penalty = 1.5 \quad x = 50\% \quad equal\_edges = count(indidvidual1 == individual2) \text{ (adjency representation)} \quad (2)$$

The scaling is increased on the **likeness of the directed edges** of the other solutions within the population. The directed edges since the problem is predominately asymmetric. The scaling is defined as such that the actual distance is the lower bound of the fitness: $1 \leq scaling$. This mechanism uses parameters in its equation to calculate the scaling values. Both the threshold for applying the scaling as well as the penalty factor for the likeness can varied to increase or decrease the punishment on likeness and increase or decrease diversity. No attempt has been made to make these parameters dynamic; for this, we refer to future work. The likeness penalty has shown a great effect on allowing new solutions that are vastly different from others to be accepted by the population.

3.12 Stopping criterion

There is a **stagnation criterion** that triggers when the best individual does not improve for a set number of iterations. For the second Process, this is after 10% of the expected iteration per minute, and for the genetic core, this is after 30% of the expected iteration per minute. The overall stopping criteria is the time limit imposed on the problem of 300 seconds.

3.13 Parameter selection

Most of the parameters have been chosen through empirical results. It would be remiss to call this a proper hyperparameter search as it was far from covering all of the search space. However, most parameters were shown **empirically** to be selected properly through iterations and common sense. The remaining parameters were chosen as "industry standard", for example, the exponential decay for the pheromone matrix in section 3.15. More information on the parameters can be found in section 4.1.

3.14 Elitism

A section of the population was shielded from the elimination phase and used for local optimisation. This population, called 'immune' in previous sections, consists of **all converged populations and a fixed amount of non-converged solutions currently being optimised**. This immunity depends on the converged solutions as $immunity = converged + base\_immunity$ .To limit its growth a reset of the converged population was implemented on the condition $converged > 3 * base\_immunity$.

3.15 Pheromone mechanism

Inspired by my course about emergence in complex systems form the Athens course and [Marco Dorigo 1997][Dorigo and Gambardella 1997], we implemented a pheromone offspring generation mechanism. The concept behind the pheromone offspring generation is **generational knowledge**. The pheromone matrix does not only contain information from two parents but from the entire population and all previous generations as well.

Every offspring ever generated, even those that did not survive the elimination, contributed to the pheromone matrix as follows:

$$\forall x \in offspring\ , \forall edge \in x\ :$$
$$pheromone\_matrix[edge] += 0.95 * pheromone\_matrix[edge] + \frac{1000 * number\_of\_cities}{distance} \quad (3)$$
$$\text{with } distance = \sum distance\_matrix[edge]\ \ \forall edge \in x$$

Every local decision of the next edge now has been influenced by the paths already constructed by all the individuals who came before. The matrix has an exponential decay to ensure the dissipation of originally bad decisions. No attempt has been made to make this parameter dynamic, nor have there been any experiments to determine the best value of exponential decay.

Offspring generated by the offspring matrix is **constructed stochastically** using

$$probabilities = 0.2 * probabilities\_distances + 0.8 * probabilities\_pheromome \tag{4}$$

with

$$probabilities\_distances[previous\_city][next\_city] = \frac{\sum \frac{1}{distanceMatrix[previous\_city]}}{distanceMatrix[previous\_city][next\_city]} \tag{5}$$

$$probabilities\_pheromone[previous\_city][next\_city] = \frac{pheremone\_matrix[previous\_city][next\_city]}{\sum pheremone\_matrix[previous\_city]} \tag{6}$$

The weight was picked arbitrarily. For optimisation of these parameters we refer to future work.

3.16 Concurrent and parallell implementation

A **concurrent process** was implemented that follows the following algorithm:[*multiprocessing* n.d.]

1. local optimise until convergence using 3-opt
2. reduce the population back to the original size
3. created inversion mutation on converged population
4. check for stagnation and reset population using pheremone matrix

In this second core, the local optimise step was parallellised using threads.

The first core has numerous variation mechanisms. We used threads to parallelize all of them. However, on bigger problems, we saw that there were massive fluctuations in run time for the variation process. This was also noticeable in the total time as it ranged from around 1 second to one minute. Sequential operations performed worse than the minimal time of threading but significantly better than the average and worse time of threading. Although hard to be sure, we suspect this to be the case due to **cache misses**. When switching threads, the cache is unable to keep storing the information of the other threads. This means that when we want to switch back, the cache needs to be refilled by the OS with the correct data, taking a lot of time. Other evidence also points towards cache misses: sometimes, the delay was between function calls in the lowest function written. After significant time spent trying to alleviate the issue, it was decided to restore sequential computing on the first process.

4 Numerical experiments

4.1 Metadata

4.1.1 Constant parameters

$$k\_selection : 5 \tag{7}$$
$$k\_elimination : 30 \tag{8}$$
$$random\_initialisation : 30\% \tag{9}$$
$$NN\_initialisation : 30\% \tag{10}$$
$$Greedy\_initialisation : 40\% \tag{11}$$
$$OX1\_offspring : 2 \tag{12}$$
$$POS\_offspring : 2 \tag{13}$$
$$Heuristic\_offspring : 2 \tag{14}$$
$$DM\_subtour\_length : 3 \tag{15}$$
$$SIM\_subtour\_length : 3 \tag{16}$$
$$IVM\_subtour\_length : 3 \tag{17}$$
$$number\_of\_parent\_pairs : 3 \tag{18}$$
$$max\_number\_of\_mutations : 6 \tag{19}$$
$$mutation\_rate : 90\% \tag{20}$$
$$likenes\_penalty : 1.5 \tag{21}$$
$$weigth\_pheromones : 80\% \tag{22}$$
$$max\_converged : 3 * base\_immunity \tag{23}$$

4.1.2 dynamic parameters / self-adaptable parameters

$$population\_size : [base\_population\_size, base\_population\_size + max\_immunity] \tag{24}$$
$$immunity : [base\_immunity, base\_immunity + max\_convergence] \tag{25}$$
$$restriction\_length\_inversion : [3, \inf] \tag{26}$$

4.1.3 Tour specific parameters

Second_core_reset and sync_pheromone_matrix are fractions of the expected iterations per minute.

parameters	percentage	tour50.csv	tour100.csv	tour200.csv	tour500.csv	tour1000.csv
population_size_second_process		10	10	10	3	3
base_immunity		5	5	5 3	3	
base_population_size		80	80	80	60	60
pheromone_offspring		15	15	15	10	10
OX1_subpath_length	15%	8	15	30	75	150
POS_number_of_fixed_positions	15%	8	15	30	75	150
likeness_elimination_initialisation	95%	48	95	190	475	950
range_NN_initialisation	5%	3	5	10	25	50
threshold_likeness_penalty	50%	25	50	100	250	500
second_core_reset	10%	150	28	10	8	6
sync_pheromone_matrix	30%	750	180	36	30	15

4.1.4 Computer system

Processor:	12th Gen Intel(R) Core(TM) i7-1255U	Total physical memory:	15.7 Gb
Clock speed:	1700 Mhz	Available physical memory:	448 Mb
Cores:	10	Total virtual memory:	30.8 Gb
Logical Processors:	12	Available virtual memory:	3.9 Gb
		python:	3.12.3

4.2 tour50.csv

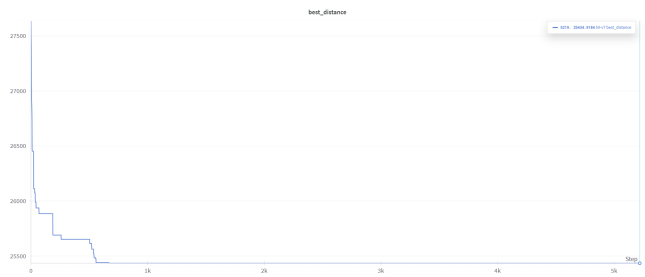

Figure 2: Best objective value

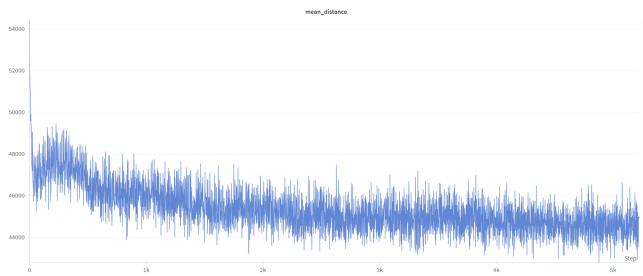

Figure 3: Mean objective value

Best tour length: 25434.9184

Sequence of cities (adjacency representation):

$$[26, 43, 31, 19, 15, 8, 22, 41, 46, 48, 13, 3, 27, 7, 1, 47, 4, 40, 24, 38, 18, 32, 49, 44, 45, \quad (27)$$

$$12, 5, 28, 36, 37, 9, 29, 0, 34, 21, 16, 23, 39, 10, 25, 33, 14, 11, 30, 17, 2, 35, 42, 6, 20] \quad (28)$$

The algorithm performs enough iterations to allow enough randomisation. Mechanisms combat the reduction in population diversity caused by the local search operators. The quality of the solution is good, as the algorithm needs numerous iterations of randomness before finding a slight increase in distance, indicating that we are near the overall optimum.

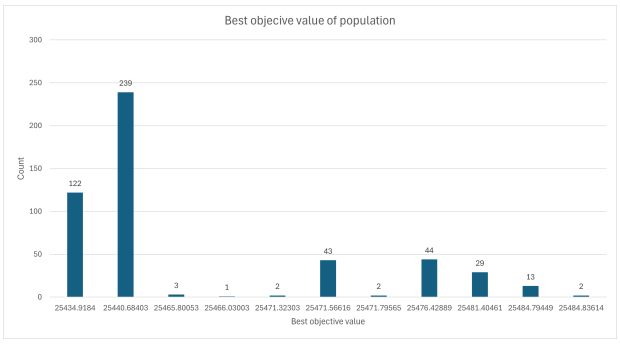

Figure 4: Best objective value

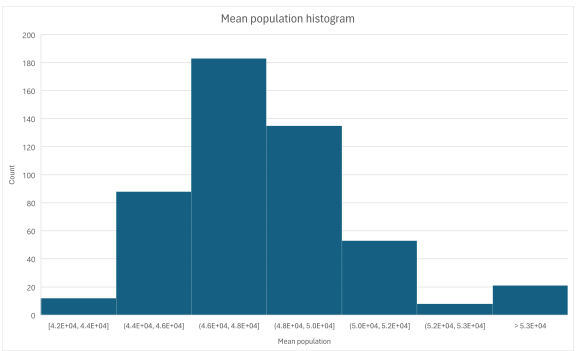

Figure 5: Mean objective value

An experiment was run solving the $50tour.csv$ problem 500 times with a time limit of 5 minutes for each iteration. Figures 4 and 5 show the result of the best and mean of the final population respectively. We see an overal good performance of the algorithm 361 runs ending with a result of either 25440 or 25434. The average best objective value is 25449.21 with a estimated standard deviation for the population using the experiment as sample of 17.25364. The mean of the population after 500 iteration indicates a assymptotic behaviour of a normal distribution with a long tail to infinity. The average is 48024.22 with a estimated standard deviation of 4695.947.

4.3 tour100.csv

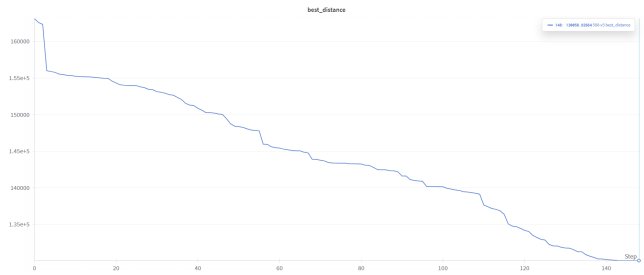

Figure 6: Best objective value

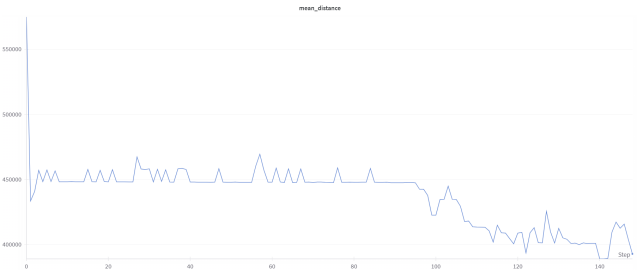

Figure 7: Mean objective value

Best tour length: 78201.26345

The results of running the algorithm on the problem with 500 cities are shown in figures 6 and 7. The algorithm performs enough iteration to allow enough randomisation. Mechanisms are combatting the reduction in population diversity caused by the local search operators. The quality of the solution is good, as the algorithm needs numerous iterations of randomness before finding a slight increase in distance, indicating that we are near the overall optimum.

4.4 tour500.csv

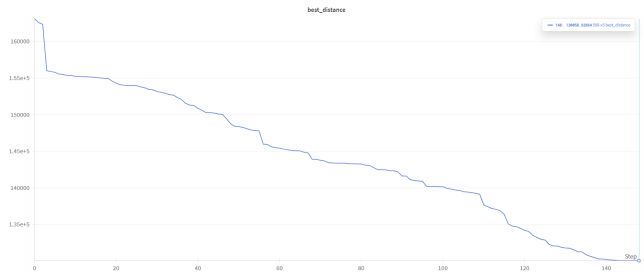

Figure 8: Best objective value

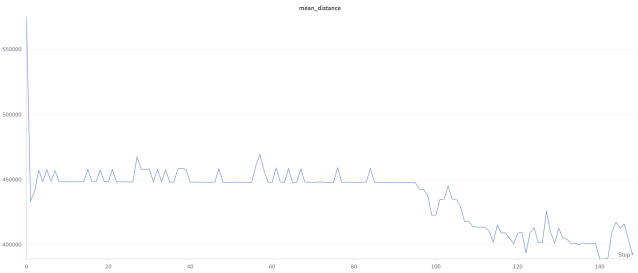

Figure 9: Mean objective value

Best tour length: 130058.92664

The results of running the algorithm on the problem with 500 cities are shown in figures 8 and 9. The algorithm performs enough iteration to allow enough randomisation. Mechanisms are combatting the reduction in population diversity caused by the local search operators. We believe the quality of the solution to be good, as the algorithm reaches a plateau right before the end while decreasing in a linear fashion until that point.

4.5 tour1000.csv

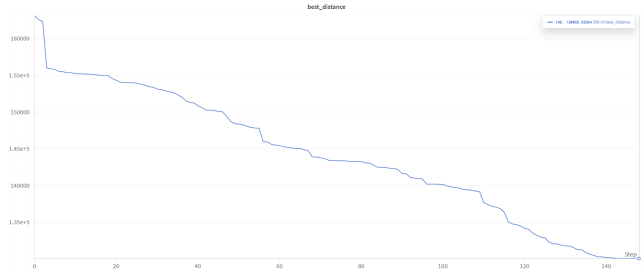

Figure 10: Best objective value

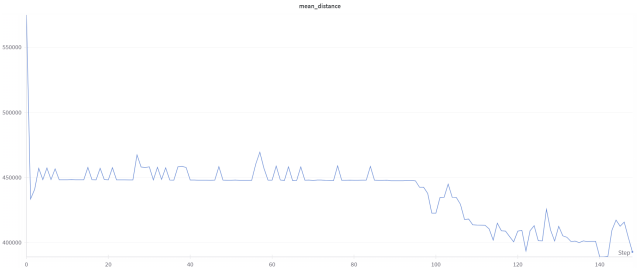

Figure 11: Mean objective value

Best tour length: 160408.85827

The results of running the algorithm on the problem with 500 cities are shown in figures 10 and 11. The algorithm performs few iterations compared to other problems. We see a sudden drop in best objective value as we suddenly come across a mutation we could locally optimise much further. The quality of the solution is only decent, as the algorithm is still in a linear decent when the time runs out.

5 Critical reflection

5.1 Strengths of evolutionary algorithms

1. It is an **iterated process**. There is a trade-off between possible better solutions and faster results. It gives you approximated solutions at any point, while other algorithms might compute until the final result is known, with no approximation in between.
2. It works great when there is little known about the best solution and a vast solution space to be considered. The tradeoff between exploration and exploitation makes it perfect to **explore the search space**. For example, hyperparameter tuning for another algorithm.
3. The general approach can be applied to most problems that are discrete in nature, or discretisation of problems. This **flexibility** of the problem is a unique property of genetic algorithms. This is why the genetic algorithms are suited to handle NP problems, for which no specific algorithm is known.

5.2 Weaknesses of evolutionary algorithms

1. The **stopping criteria** are hard to define. You can never be sure that your algorithm will never find a better solution. You can never be sure of absolute convergence.
2. It is **hard to design** a performant evolutionary algorithm. There are a lot of things that you have to take into consideration and possible improvements. Designing problem-appropriate mechanisms is not easy and often relies on heuristics instead of mathematical properties.
3. In the beginning you can still understand what is going on and why your algorithm is behaving a certain way. However, after a while with increasing complexity, it becomes hard to fully understand the impact that a certain change or addition has. Since all changes interact with almost all of the algorithms, sometimes over generations, **it becomes impossible to understand why certain changes cause certain effects.**

5.2.1 Main lessons

We belief genetic algorithms to be a powerful tool for problems that require an **iterative algorithm and lent toward approximated solutions**. These problems include most real-world scenarios, as often the model of the problem is very complex, and exact solutions are impossible to find or verify. Genetic algorithms can shine in finding approximated solutions for NP problems, where the optimal solution cannot easily be verified anyway; hyperparameter tuning, where the search space is vast and discrete; and approximated problems in general. However, it **takes great care** and iterations of trial and error to develop relevant heuristics for these algorithms. Without them, the genetic algorithm will never find a performant solution.

We do believe genetic algorithms lent themselves to solving the travelling sales problem as the tsp problem is NP-hard and will always produce an approximated solution. However, there are other approaches that are also prominent like simulated annealing. How the performance is evaluated depends mainly on the implementation of the program and the **heuristics** used. Computational cost is a big issue with GA's as the power lies within an **excessive amount of iterations**, which is why Python might not be the preferred language to develop them in.

What surprised me the most was **the outcome of the recombination operators**. The heuristics employed make a lot of sense to me; often however, the offspring was worse than the parents. This shows how difficult it is to develop a performant genetic algorithm. Similarly, since everything is connected it is hard to measure the contribution of a single aspect of the algorithm and even harder to understand why results are the way they are.

Overall we belief we have learned a great deal during the course of this project. We gained additional experience in algorithm design and a newly found interest in stochastic algorithms.

6 Generative AI compliance form

No Generative AI was used during this project.

References

David S. Johnson, Lyle A. McGeoch (Nov. 1997). "Genetic Algorithms for the Travelling Salesman Problem: A Review of Representations and Operators". In: *Local Search in Combinatorial Optimization*, pp. 215–310. URL: https://www.cs.ubc.ca/~hutter/previous-earg/EmpAlgReadingGroup/TSP-JohMcg97.pdf.

Dorigo, Marco and Luca Maria Gambardella (1997). "Ant colonies for the travelling salesman problem". In: *Biosystems* 43.2, pp. 73–81. ISSN: 0303-2647. DOI: https://doi.org/10.1016/S0303-2647(97)01708-5. URL: https://www.sciencedirect.com/science/article/pii/S0303264797017085.

Larranaga, Pedro et al. (Jan. 1999). "Genetic Algorithms for the Travelling Salesman Problem: A Review of Representations and Operators". In: *Artificial Intelligence Review* 13, pp. 129–170. DOI: 10.1023/A:1006529012972.

Marco Dorigo, Luca Maria Gambardella (Apr. 1997). "Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem". In: *IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION* 1. URL: https://people.idsia.ch/~luca/acs-ec97.pdf.

multiprocessing (n.d.). URL: https://docs.python.org/3/library/multiprocessing.html.

Numba documentation (n.d.). URL: https://numba.pydata.org/.